Real-Time Contour Extraction and Object Detection

A Frame-by-Frame Performance Whitepaper

Task 4: Computer Vision Object Detector & Frame-by-Frame Segmenter Pipeline

AI Internship Report | Author: FIZA ASGHAR | Date: 13 OCTOBER 2026

Abstract

This paper evaluates two real-time computer vision pipelines that detect and track objects in video: (A) a classical OpenCV pipeline built from a Gaussian filter, an adaptive threshold, pixel-bound contour filtering, shape classification and centroid tracking, and (B) a pre-trained YOLOv8n deep-learning detector with ByteTrack tracking. Both overlay a dynamic per-class count vector on every frame, and every frame is timed. On a synthetic video with known ground truth, pipeline A reached a Macro F1 of 0.990 at about 248 FPS on a CPU, with no identity switches. An ablation study shows that the adaptive threshold, the Gaussian filter and the pixel bounds each matter: removing them lowered Macro F1 to 0.763, increased runtime 2.7-fold, and lowered precision to 0.27 respectively. On a 650-frame video made from ten photos of a real room, YOLOv8n ran at about 40 FPS on a Tesla T4 GPU but produced detections in only 35% of the frames and several confident mistakes (for example, a ceiling fan labelled "airplane" with 0.94 confidence). The results show a clear trade-off between the speed and transparency of the classical pipeline and the semantic breadth, and the limits, of a pre-trained detector.

1. Objective and Scope

The task is to build a pipeline that extracts and tracks target contours in real time, using OpenCV or a pre-trained deep-learning model. Requirements covered in this study: (i) Gaussian filtering and adaptive thresholding on video streams, (ii) shape extraction based on pixel bounds, (iii) dynamic text count vectors overlaid on target classes, and (iv) frame-by-frame inference metrics. Both pipelines were implemented in Python and evaluated with the same metric definitions.

2. System Design

2.1 Pipeline A: classical OpenCV

Each frame passes through the stages below. The parameters were set once and kept fixed for all experiments.

Stage	Operation and parameters
1. Grayscale	BGR to gray conversion.
2. Gaussian filter	5x5 Gaussian blur to suppress sensor noise before thresholding.
3. Adaptive threshold	Gaussian-weighted local mean, neighbourhood 41 px, constant C = 10, inverted binary output; followed by a 3x3 morphological close.
4. Contours + pixel bounds	External contours only; a contour is kept if its area lies between 250 and 20,000 pixels (removes speckle noise and merged or background blobs).
5. Shape classification	Circularity 4*pi*A/P^2 >= 0.84 gives circle; otherwise the polygon approximation (tolerance 4.5% of perimeter) with 3 vertices gives triangle and 4 vertices gives rectangle; anything else is labelled "other".
6. Tracking	Greedy nearest-centroid tracker (maximum jump 50 px, a track survives 5 missed frames) gives each contour a persistent ID and a motion trail.
7. Overlay	Contours, IDs and trails, plus a panel with the count vector [circle, triangle, rectangle], total objects, FPS and latency.

2.2 Pipeline B: YOLO

A pre-trained YOLOv8n model (80 COCO classes) was run frame by frame with confidence threshold 0.35, input size 640 and ByteTrack tracking, on a Tesla T4 GPU in Google Colab. The overlay shows boxes with track IDs, a count of the five most frequent classes in the current frame, FPS and latency.

2.3 Test videos

- **Synthetic video (pipeline A):** 300 frames at 640x480 with 7 moving shapes (3 circles, 2 triangles, 2 rectangles) on a background with uneven lighting, Gaussian noise and 20 flickering speckles per frame.

Shapes bounce off walls and each other. The exact position and class of every shape is known, which allows true accuracy to be measured.

- **Room video (pipeline B):** 650 frames at 1280x720 (26 s) assembled from ten photos of a real room (notebook, stool, bed, calendar, wall frame, ceiling fan, air conditioner, wall hangings, earbuds case, charger), each shown for 2.6 s with a slow zoom and cross-fades. It is a slideshow of still photos, not a real camera recording, and has no bounding-box ground truth.

2.4 Metrics

Per frame: latency (ms) and FPS, time per stage, objects detected and the per-class count vector. With ground truth: detections are matched to true shapes by centroid distance (22 px), from which per-class precision, recall and F1, the exact-count accuracy of the count vector, the mean absolute count error and the number of identity switches are computed. Runtime values are wall-clock and depend on hardware; counts and accuracy are deterministic.

3. Results: Pipeline A (Classical OpenCV)

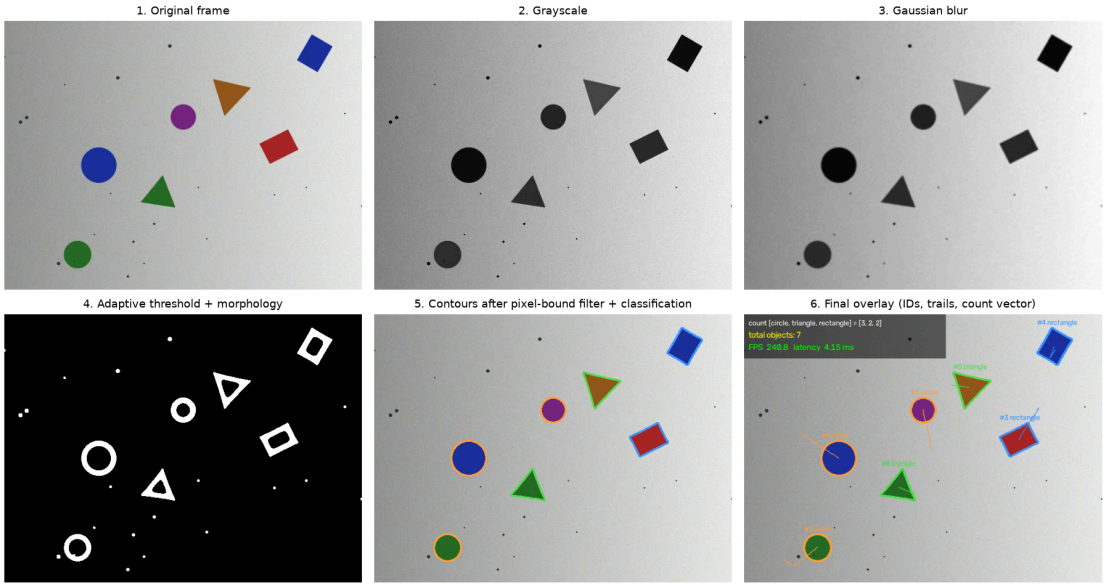


Figure 1: Pipeline stages on one frame. Large uniform shapes appear as outlines after adaptive thresholding; external contours still recover the full shape.

Detection quality. Across 2,100 true shape instances, pipeline A reached Macro F1 0.990 and reproduced the exact per-class count vector in 92.3% of the frames. Seven tracks were created for the seven shapes, with zero identity switches. The few errors are 22 missed circles (18 read as rectangles, 4 as "other") and 19 false rectangle detections, which are mostly those same circles.

Table 1: Detection quality against ground truth (300 frames)

Class	Precision	Recall	F1	TP	FP	FN	Count MAE
Circle	1.000	0.976	0.988	878	0	22	0.073
Triangle	1.000	0.998	0.999	599	0	1	0.003
Rectangle	0.969	1.000	0.984	600	19	0	0.063
Macro average	**0.990**	**0.991**	**0.990**				

Speed. Mean latency was 4.04 ms per frame (median 3.77 ms, 95th percentile 5.71 ms, maximum 8.99 ms), a processing speed of 247.5 FPS; including drawing and writing the output video the end-to-end speed was 90.3 FPS. The adaptive threshold dominates the cost (3.20 ms of 4.03 ms, 80%), while grayscale conversion, blur, contour analysis and tracking together take under 1 ms.

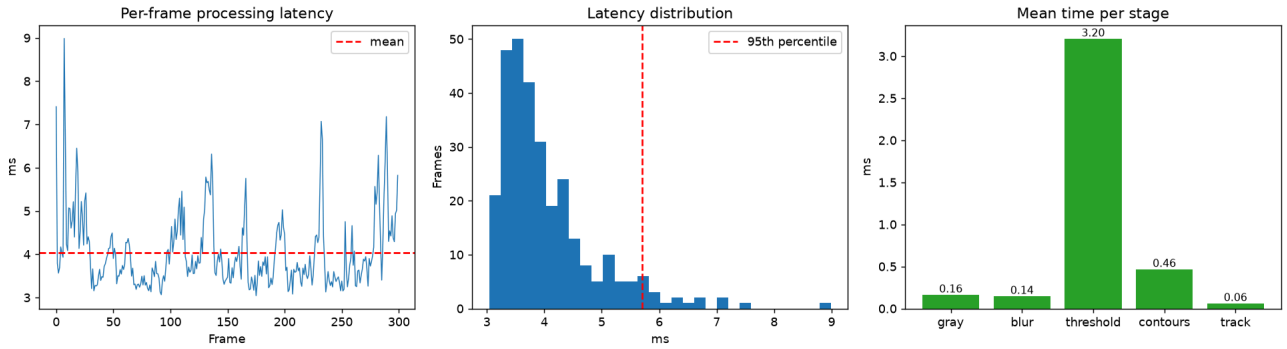


Figure 2: Per-frame latency, its distribution and the mean time per stage.

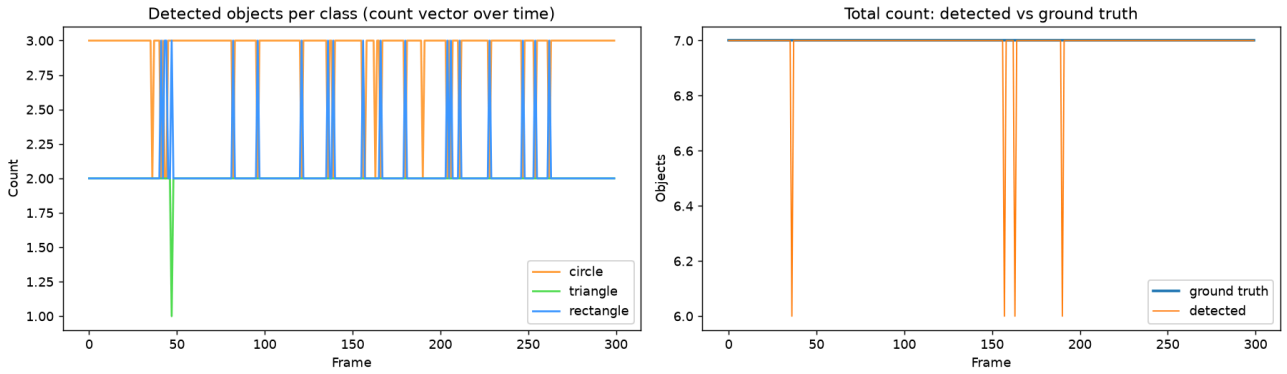


Figure 3: Count vector over time (left) and total detected vs true count (right). The detected total equals the true total of 7 in all but four frames.

3.1 Ablation study

Each key component was removed in turn and the whole video was processed again (timing here was measured without drawing, which is why the full pipeline is faster than in Section 3).

Table 2: Ablation of pipeline A

Variant	Macro F1	Loc. precision	Loc. recall	Latency (ms)	FPS
Full pipeline	**0.990**	**1.000**	**1.000**	**3.34**	**299**
No Gaussian blur	0.990	1.000	1.000	8.97	112
Global Otsu threshold	0.763	1.000	0.643	1.21	830
No pixel bounds	0.639	0.273	1.000	4.60	217

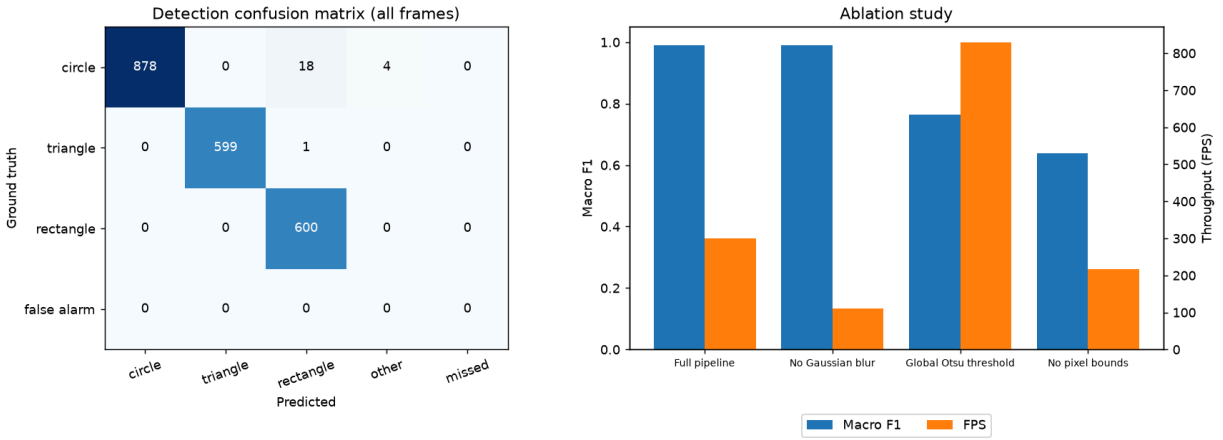


Figure 4: Detection confusion matrix (left) and ablation study (right).

- **Adaptive vs global threshold:** a single global (Otsu) threshold missed over a third of the shapes (recall 0.643) because the background brightness changes across the frame. The local threshold copes with this; the global one is faster because a single threshold is cheaper to compute.

- **Gaussian filter:** accuracy did not change, but runtime rose 2.7-fold, most likely because unfiltered noise creates many tiny contours that must be examined and rejected. On this video the filter therefore acts mainly as a speed measure; it may also protect accuracy in noisier footage.
- **Pixel bounds:** without the area limits every speckle became an "object" and precision fell from 1.000 to 0.273, the largest quality loss of all variants.

4. Results: Pipeline B (YOLOv8n)

Speed. The first frame took 938 ms (GPU warm-up) and should be excluded from benchmarks. For the remaining 649 frames, latency was 24.8 ms on average (median 23.8 ms, 95th percentile 34.9 ms), about 40 FPS. Over all 650 frames the mean was 26.2 ms (38.2 FPS). Mean preprocessing, model inference and post-processing took 1.55, 9.48 and 1.09 ms; the remaining 12 to 13 ms per frame (not measured separately) includes tracking, data transfer and Python overhead.

Detections. The model produced 271 detections, 0.42 per frame on average (maximum 3), with a mean confidence of 0.526. In 421 of 650 frames (64.8%) nothing was detected. Detections by class: person 100, airplane 61, dining table 49, laptop 46, bed 4, cell phone 4, chair 3, book 3, refrigerator 1. The mean absolute change of the object count between consecutive frames was 0.11.

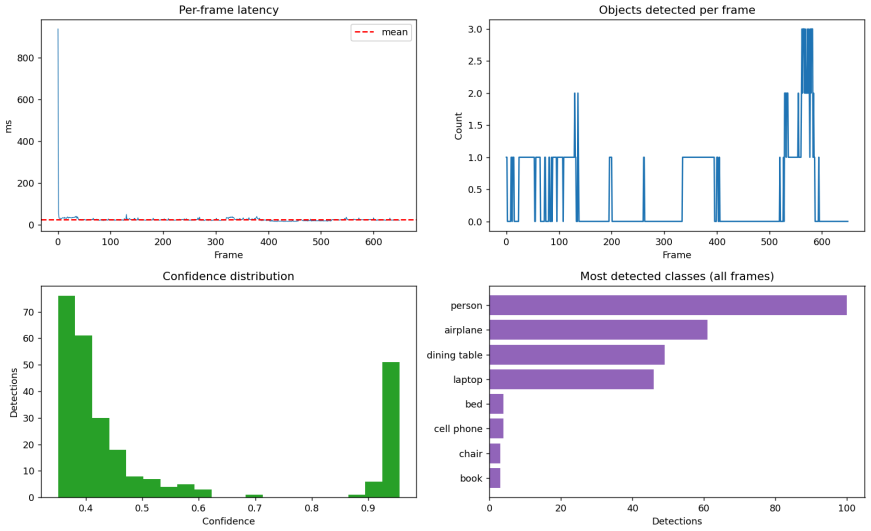


Figure 5: YOLOv8n on the room video: latency, objects per frame, confidence distribution and most detected classes.

Error analysis. Because the room video has no annotated boxes, mean average precision cannot be computed. Instead, the label given for each photo was assessed manually against what the photo actually shows (Table 3). This is a qualitative judgement by the author, not a benchmark score.

Table 3: What YOLOv8n detected in each photo (65 frames per photo)

Photo (real object)	Detected as	Frames with detection	Mean confidence	Assessment
Notebook	laptop	46 / 65	0.47	Wrong (should be book)
Stool	dining table	45 / 65	0.38	Plausible (small table)
Bed and keyboard	person, table, chair	5 / 65	0.46	Bed and keyboard missed
Calendar on side table	bed (edge visible)	5 / 65	0.40	Partly plausible
Wall frame	book	2 / 65	0.41	Not a COCO class
Ceiling fan	airplane	55 / 65	0.94	Wrong, with high confidence
Air conditioner	airplane, cell phone	10 / 65	0.60	Wrong (not a COCO class)
Wall hangings	none	0 / 65	-	Not a COCO class
Hand with earbuds case	person	58 / 65	0.40	Hand read as person; case missed
Charger	person, book	3 / 65	0.38	Not a COCO class

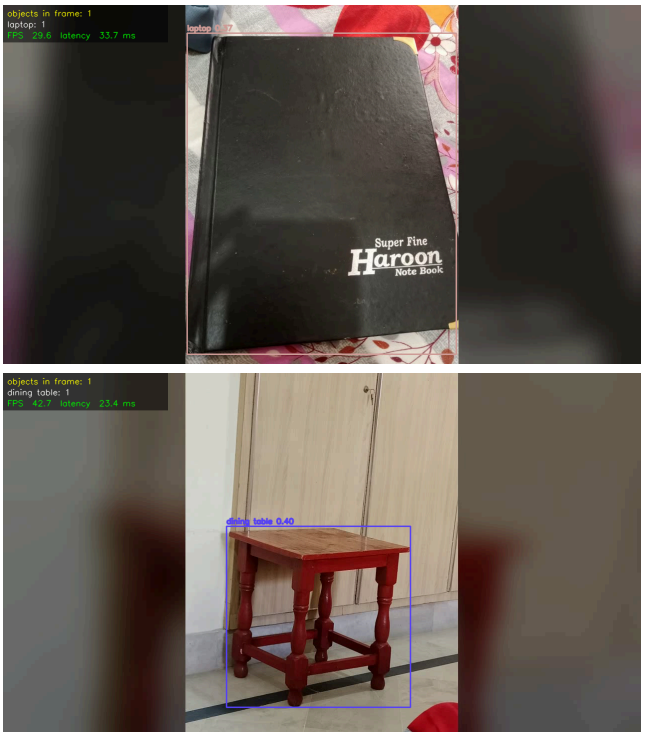


Figure 6: Annotated frames 10 (notebook labelled "laptop", 0.37) and 120 (stool labelled "dining table", 0.40) with the live count panel.

5. Comparison and Discussion

	A: Classical OpenCV	B: YOLOv8n
Output	Shape class (circle / triangle / rectangle) plus contour	Object class (COCO, 80 classes) plus box
Speed (this study)	about 248 FPS, CPU, 640x480	about 40 FPS, Tesla T4 GPU, 1280x720
Accuracy (this study)	Macro F1 0.990 on synthetic video with ground truth	No ground truth; 2 of 10 photos plausible, 1 confident error
Strengths	Very fast, transparent, no training data, exact contours	Understands real objects, no manual tuning per scene
Weaknesses	Needs contrast and a plain scene; fixed parameters; cannot name objects	Limited to 80 classes; confident mistakes; needs a GPU for real-time; boxes only

- **Speed is not comparable like-for-like:** the pipelines ran on different hardware and inputs, so the numbers show the order of magnitude of each approach rather than a fair race.
- **Confidence is not correctness:** the fan-as-airplane error had the highest confidence (0.94) of all detections, so a confidence threshold alone cannot remove such mistakes.
- **Out-of-vocabulary objects:** when an object is not among the 80 classes (calligraphy frame, air conditioner, charger), YOLO either stays silent or picks the nearest-looking class. The classical pipeline has no such vocabulary problem because it only describes shape, but it cannot say what the object is.
- **Complementary use:** a classical stage can propose regions cheaply and a neural classifier can name them, combining the strengths of both.

6. Limitations

- The synthetic video is deliberately clean (plain background, high contrast, no occlusion), so the near-perfect score of pipeline A is optimistic for real scenes. A centroid tracker would also fail under heavy occlusion.
- The room video is a slideshow of still photos. Natural motion, motion blur and temporal stability could not be tested, and the count-change statistic mainly reflects the cuts between photos.
- No annotated boxes exist for the room video, so YOLO accuracy is a manual, qualitative assessment, and mAP or IoU were not computed.
- Each experiment is a single run on one machine; only YOLOv8n at a fixed 0.35 confidence threshold was tested, and thresholds of pipeline A were not tuned automatically.

7. Conclusion

Both pipelines meet the task requirements: contours are extracted and tracked frame by frame, a per-class count vector is overlaid, and per-frame inference metrics are measured. The classical pipeline is fast (about 4 ms per frame) and accurate in a controlled scene, and its ablation shows that adaptive thresholding, filtering and pixel bounds each contribute. The pre-trained detector reaches real-time speed on a GPU and recognises real objects, but in a cluttered room it missed most items and made confident mistakes, which highlights the need for annotated evaluation data and, for custom objects, fine-tuning. Future work: annotate a small set of real video frames to compute mAP, compare larger YOLO models and fine-tune on household classes, test real camera footage with motion, and use a Kalman-filter tracker.

Appendix: Deliverables

- cv_pipeline.py (pipeline A: synthetic video generation, pipeline, tracker, metrics, ablation, figures) and task4_yolo_colab.ipynb (pipeline B, Google Colab).
- Outputs: annotated videos, frame_metrics.csv, summary.json, ablation.csv, yolo_frame_metrics.csv, yolo_summary.json and all figures.